

Setting up your workspace in WSL for ROS2 Jazzy and Turtlebot3

- Open a WSL terminal on your device and run the following command:

```
mkdir -p /home/c2irr10/turtlebot3_ws/src
```

(This will create the “turtlebot3_ws” workspace)

- Place the file “Dockerfile” file from [Simulation Files](#) inside of the turtlebot3_ws folder.

(To do this you can open your windows file explorer, scroll down to Linux on the left side bar, select the Ubuntu-24.04-> home -> c2irr10 -> turtlebot3_ws, then you can drag and drop the “Dockerfile” file inside here)

- Open a new WSL terminal and run the following commands inside to make sure that permissions are assigned correctly to edit the files inside of your workspace:

```
sudo chown -R $USER:$USER /home/c2irr10
chmod -R 775 /home/c2irr10
```

- You can now close all the open terminals.
- Open Docker Desktop application and while having it open in the background, open a new WSL terminal and run the following commands to create the docker image for Robot Operating System (ROS2 Jazzy) and Turtlebot3 default packages:

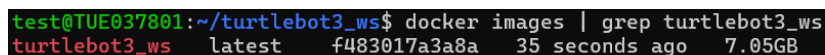
```
cd /home/c2irr10/turtlebot3_ws/
docker build -t turtlebot3_ws .
```

(this might take a few minutes)

- Make sure the docker image has been built correctly by running the following command:

```
docker images | grep turtlebot3_ws
```

You should see something like this if it was installed correctly:



```
test@TUE037801:~/turtlebot3_ws$ docker images | grep turtlebot3_ws
turtlebot3_ws latest f483017a3a8a 35 seconds ago 7.05GB
```

- After you finish building the docker image run the following command to start-up the docker container that has installed all ROS2 Jazzy and Turtlebot3 default packages:

```
docker run --rm -it --name turtlebot3_container --net=host -e DISPLAY=$DISPLAY -v /tmp/.X11-unix:/tmp/.X11-unix -v /home/c2irr10/turtlebot3_ws:/ws --user $(id -u):$(id -g) turtlebot3_ws bash
```

- In the same terminal run the following command:

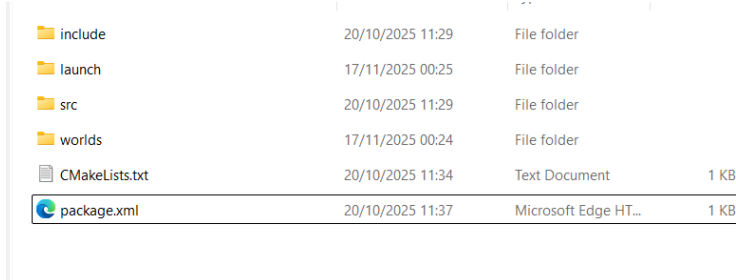
```
cd /ws/src
```

- Then run the following command to create ROS2 custom package:

```
ros2 pkg create my_tb3_world \
--build-type ament_cmake \
--dependencies launch launch_ros ament_cmake
```

(This will create a package for you to have your custom files for the world and launch simulations)

- Navigate into your new package by running the following command in the same terminal:
`cd /ws/src/my_tb3_world`
- Create two new folders, one for the world file and one for the launch files by running the following command:
`mkdir -p launch worlds`
- Inside of your Linux subsystem now if you enter the following directory (it should look similar to this): “Ubuntu-24.04\home\c2irr10\turtlebot3_ws\src\my_tb3_world” you should see 2 new folders, “worlds” and “launch”:



include	20/10/2025 11:29	File folder	
launch	17/11/2025 00:25	File folder	
src	20/10/2025 11:29	File folder	
worlds	17/11/2025 00:24	File folder	
CMakeLists.txt	20/10/2025 11:34	Text Document	1 KB
package.xml	20/10/2025 11:37	Microsoft Edge HT...	1 KB

- Download the “new_world.world” file from [Simulation Files](#) on your host device and drag and drop the file inside of the “worlds” folder that you created.
- Download the “new_world.launch.py” file from [Simulation Files](#) on your host device and drag and drop the file inside of the “launch” folder that you just created.
- Inside of the “Ubuntu-24.04\home\c2irr10\turtlebot3_ws\src\my_tb3_world” directory:
 1. delete the “CMakeLists.txt” and “package.xml” files, and then
 2. download the “CMakeLists.txt” and “package.xml” files from [Simulation Files](#) and drag and drop them into “Ubuntu-24.04\home\c2irr10\turtlebot3_ws\src\my_tb3_world” directory.
- Close all the open terminals”.
- Now the setup is finished we can try to run the simulation.

Run the Gazebo (the 3D simulation environment) custom world simulation

1. Open the Docker Desktop application on your host device.
2. Open a WSL terminal and start the Docker container by running the following command:

```
docker run --rm -it --name turtlebot3_container --net=host -e DISPLAY=$DISPLAY -v /tmp/.X11-unix:/tmp/.X11-unix -v /home/c2irr10/turtlebot3_ws/ws --user $(id -u):$(id -g) turtlebot3_ws bash
```
3. Now build and run the simulation using the following commands in order:
(Note: Make sure that all commands are run sequentially!)

```
cd /ws
source /opt/ros/jazzy/setup.bash
source /opt/turtlebot3_ws/install/setup.bash
colcon build
source install/setup.bash
export TURTLEBOT3_MODEL=burger
ros2 launch my_tb3_world new_world.launch.py
```

If the following error occurs:

```
[gazebo-2] qt.qpa.xcb: could not connect to display :0
[gazebo-2] qt.qpa.plugin: Could not load the Qt platform plugin "xcb" in "" even though it was found.
[gazebo-2] This application failed to start because no Qt platform plugin could be initialized. Reinstalling the application may fix this problem.
```

```
[create-3] [INFO] [1771923893.569709337] [ros_gz_sim]: Requesting list of world names.
[robot_state_publisher-5] [INFO] [1771923893.578868501] [robot_state_publisher]: Robot initialized
[gazebo-2] qt.qpa.xcb: could not connect to display :0
[gazebo-2] qt.qpa.plugin: Could not load the Qt platform plugin "xcb" in "" even though it was found.
[gazebo-2] This application failed to start because no Qt platform plugin could be initialized. Reinstalling the application may fix this problem.
[gazebo-2] Available platform plugins are: eglfs, linuxfb, minimal, minimalegl, offscreen, vnc, xcb.
[gazebo-2] Stack trace (most recent call last):
```

To fix this issue, change the docker command from:

```
docker run --rm -it --name turtlebot3_container --net=host -e DISPLAY=$DISPLAY -v /tmp/.X11-unix:/tmp/.X11-unix -v /home/c2irr10/turtlebot3_ws:/ws --user $(id -u):$(id -g) turtlebot3_ws bash
```

To:

```
docker run --rm -it --name turtlebot3_container --net=host -e DISPLAY=$DISPLAY -v /home/c2irr10/turtlebot3_ws:/ws -v /mnt/wslg/.X11-unix:/tmp/.X11-unix --user $(id -u):$(id -g) turtlebot3_ws bash
```

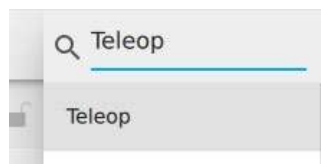
Teleop Node: Moving the Robot

To move the robot inside the simulation, we need to run the Teleop node. This can be done in two ways:

1. Use the Teleop plugin inside of the Gazebo simulation
2. Start a parallel terminal and run the node in there

1. Use the Teleop plugin inside of the Gazebo simulation

1. Inside the Gazebo app, click the : (three dots) button on the top-right toolbar.
2. In the search bar, type Teleop, then select the Teleop plugin.



3. Scroll down on the right-hand panel, and you'll find the Teleop section for robot navigation.



If you press the arrow button the robot should start moving in the simulation according to the inputs you give. You can modify the velocity values to make the robot move faster/slower.

2. Start a parallel terminal and run the node in there

Start a parallel WSL terminal after your simulation is running.

We will now attach this WSL terminal to the running docker container where the simulation is running and run the Teleop node. In order to do this, run the following commands:

(Note: Make sure that all commands are run sequentially!)

```
docker exec -it turtlebot3_container bash
cd /ws
source install/setup.bash
export TURTLEBOT3_MODEL=burger
ros2 run turtlebot3_teleop teleop_keyboard
```

You should see output similar to this:

```
root@docker-desktop:/ws# ros2 run turtlebot3_teleop teleop_keyboard

Control Your TurtleBot3!
-----
Moving around:
      w
  a    s    d
      x

w/x : increase/decrease linear velocity (Burger : ~ 0.22, Waffle and Waffle Pi : ~ 0.26)
a/d : increase/decrease angular velocity (Burger : ~ 2.84, Waffle and Waffle Pi : ~ 1.82)

space key, s : force stop

CTRL-C to quit
```

You can now move the robot by pressing the w, a, s, d, x keys.

Control the Robot

Use the following keys to move the robot:

Key	Action
W	Move forward
X	Move backward
A	Turn left
D	Turn right
S / Space	Stop
Press CTRL + C to quit.	

If both the **Gazebo Teleop plugin** and the **terminal Teleop node** are active at the same time, **only one of them will work** — the terminal node takes priority. To avoid conflicts, use **either** the plugin **or** the terminal method, not both.

Launching Rviz (ROS Visualization)

This section will guide you in opening **Rviz**, the ROS2 visualization tool used to monitor the robot's sensors, pose, and mapping progress.

RViz is a 3D interface that allows you to:

- View LiDAR scans in real time
- Track the robot's position and orientation
- Visualize maps as they are being created

Since RViz is a graphical application and you are working inside a Docker container on WSL, follow these steps to properly open it:

1. **Open a parallel WSL terminal** on your host machine.
2. **Attach to your running container by running the following command (make sure your simulation in Gazebo is open before this):**

```
docker exec -it turtlebot3_container bash
```

3. **Navigate to the workspace:**

```
cd /ws
```

4. **Sourcing:**

```
source install/setup.bash
```

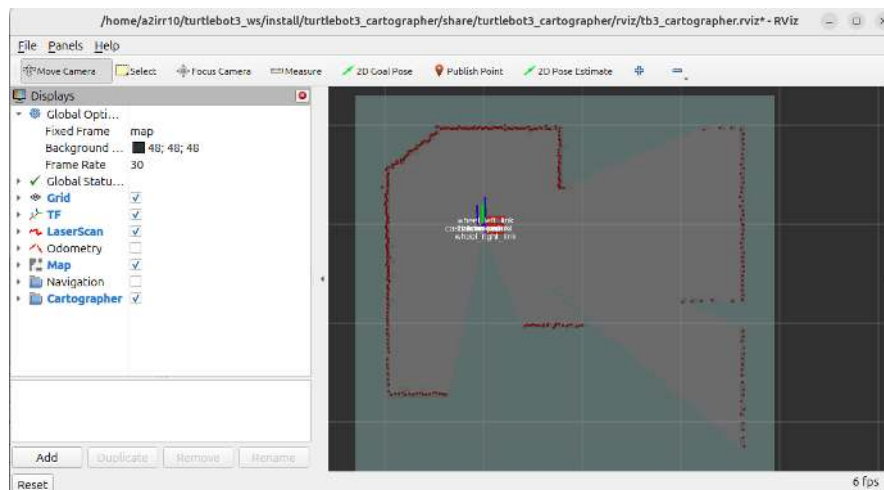
5. **Set the TurtleBot3 model:**

```
export TURTLEBOT3_MODEL=burger
```

6. **Launch RViz with Cartographer:**

```
ros2 launch turtlebot3_cartographer cartographer.launch.py use_sim_time:=True
```

You should now see a window open like this:



If you also launch the Teleop node (as shown earlier), you can manually move the robot and observe the LiDAR scans in real time as the map builds.